Reservas de Game Spot: arquitectura orientada a servicios con dos proveedores y un consumidor web

Cristian Farid Amado Peña y Cristian Santiago Silva Muñoz

Facultad de Ingeniería de Sistemas, Universidad Santo Tomás, Villavicencio

Desarrollo Orientado a Servicios

Diana Velásquez

18 de noviembre de 2026

Resumen

Game Spot es una sala de videojuegos del barrio San Jorge, en Villavicencio (Colombia), que alquila consolas por hora y por media hora. Este documento presenta la versión orientada a servicios de su sistema de reservas, construida como tres aplicaciones Spring Boot independientes que se comunican por HTTP con JSON. El proveedor 1 (puerto 8091) expone el catálogo de equipos, sus tarifas por hora y por media hora y el horario de atención. El proveedor 2 (puerto 8092) gestiona las reservas en bloques de 30 minutos y emite JSON Web Tokens. La aplicación consumidora (puerto 8080) sirve una página web hecha con HTML, CSS y JavaScript con la API Fetch, y su backend llama a los dos proveedores con Spring RestClient para coordinar dos operaciones integradas: la disponibilidad diaria de un equipo y la creación de una reserva. Cada proveedor guarda sus datos en su propia base H2 en archivo, definida con scripts SQL y accedida mediante repositorios JPA. El documento describe las reglas del negocio, la arquitectura, el contrato de la API, los objetos de transferencia de datos, el modelo de datos, el diseño de la autenticación y las pruebas: 29 pruebas automatizadas en los tres proyectos, una colección de Postman con 40 solicitudes y una prueba manual de punta a punta que incluye un proveedor detenido.

Palabras clave: arquitectura orientada a servicios, API REST, Spring Boot, RestClient, JSON Web Token, H2

Reservas de Game Spot: arquitectura orientada a servicios con dos proveedores y un consumidor web

Game Spot alquila cinco equipos: dos consolas Xbox 360, dos Xbox Series S y una sala VIP con una Xbox Series X. Cada tipo de equipo tiene una tarifa por hora y otra por media hora: 4.000 y 2.500 pesos para la Xbox 360, 5.000 y 3.000 para la Xbox Series S y 10.000 y 6.000 para la sala VIP. Los clientes reservaban por WhatsApp, así que el local no tenía una forma confiable de mostrar los turnos libres, de impedir que dos personas reservaran el mismo equipo a la misma hora ni de saber quién había pagado. El equipo ya había construido una versión monolítica del sistema para otra asignatura; este proyecto adapta el mismo negocio a los requisitos del desarrollo orientado a servicios.

Dividir el sistema en servicios se justifica porque las dos capacidades cambian por razones y en momentos distintos. El catálogo cambia cuando el dueño compra un equipo, repara uno o actualiza precios, y casi siempre se consulta. Las reservas cambian cada vez que un cliente reserva, paga o cancela, y llevan los datos del cliente y la autenticación. Separarlas permite que cada proveedor sea dueño de sus datos y evolucione por su cuenta, y la consumidora muestra cómo una aplicación cliente combina ambos a través de sus contratos públicos.

Objetivos

El objetivo general es construir una solución web para reservar los equipos de Game Spot, organizada como dos aplicaciones proveedoras y una consumidora que se comunican mediante API REST. Los objetivos específicos son (a) exponer el catálogo de equipos, tarifas y horario como un proveedor con CRUD completo, (b) exponer las reservas y la autenticación como un segundo proveedor, (c) construir una consumidora con interfaz web que integre ambos proveedores con RestClient, (d) guardar los datos de cada proveedor en su propia base relacional

con DDL, DML, repositorios y ORM, (e) proteger las operaciones que modifican datos con JWT y (f) verificar la solución con pruebas automatizadas, Postman y una prueba con un proveedor detenido.

Alcance

El alcance incluye las tres aplicaciones, el CRUD del catálogo, el ciclo de vida de la reserva (crear, pagar un adelanto simulado y cancelar), la disponibilidad diaria, la autenticación de usuarios de prueba y la interfaz web para clientes y administradores. Quedan por fuera los pagos reales en línea, las notificaciones por correo, el registro de usuarios, el despliegue en un servidor público, una base de datos de producción y el cobro de tiempo adicional fuera de los bloques reservados. Este último punto se dejó a propósito para una iteración posterior al flujo principal de reservas, como se acordó con la docente.

Reglas del negocio

La Tabla 1 resume las reglas que aplica el sistema y dónde se hace cumplir cada una. Se cerraron con la docente después de la revisión del avance: el local trabaja con un único horario fijo, el adelanto no se devuelve, una reserva sin pagar vence a los 15 minutos y el tiempo adicional se atiende después del flujo principal. La columna de estado indica si la versión actual ya hace cumplir la regla.

Tabla 1

Reglas del negocio del sistema de reservas

Regla	Definición	Dónde	Estado
Horario	Todos los días de 3:00 p. m. a 9:00 p. m. (hora de Bogotá). La reserva debe empezar y terminar dentro de ese horario.	Proveedor 1 (horarios_atencion); validación de la consumidora (400)	Implementada (CP-36g, CP-38d)
Bloques	Las reservas empiezan a las :00 o :30 y duran un múltiplo de 30 minutos, de 30 a 240, en el futuro.	Proveedor 2 (ReservaService, 400)	Implementada (CP-37c)

Regla	Definición	Dónde	Estado
Sin cruces	Un equipo no puede tener dos reservas activas (pendientes o confirmadas) que se crucen.	Proveedor 2 (consulta JPQL, 409)	Implementada (CP-37d)
Precio	Cada hora completa a la tarifa por hora y, si sobran 30 minutos, la tarifa de media hora. Xbox Series S: 30 min = 3.000; 60 min = 5.000; 90 min = 8.000.	Proveedor 2 (calcularPrecioPesos)	Implementada (CP-37b, CP-37i)
Adelanto	50 % del precio, redondeado hacia arriba. Pagarlo (simulado) cambia la reserva de PENDIENTE_PAGO a CONFIRMADA.	Proveedor 2	Implementada (CP-37f)
Adelanto sin devolución	Si se cancela una reserva confirmada, el adelanto no se devuelve. El sistema no tiene operación de reembolso: al cancelar, el adelanto pagado queda registrado.	Proveedor 2 (reservas.adelanto_pesos)	Implementada; la página pide confirmación
Cancelación lógica	Las reservas no se borran: cancelar pone CANCELADA y la fecha, y libera el bloque.	Proveedor 2	Implementada (CP-37g)
Vencimiento	Una reserva que no se paga dentro de los 15 minutos siguientes a su creación vence (EXPIRADA): libera el bloque y ya no se puede pagar ni cancelar (409).	Proveedor 2 (ReservaService, buscarActivasQueSeCruzan)	Implementada (CP-37j, CP-37k)
Tiempo adicional	Cobro del tiempo jugado después de los bloques reservados.	—	Fuera del alcance; después del flujo principal

Nota. Los códigos de prueba corresponden a la Tabla 10.

El vencimiento se calcula en lugar de guardarse. La reserva conserva el estado PENDIENTE_PAGO en la base, y el proveedor 2 la trata como vencida cuando pasan 15 minutos desde *creada_en*: la consulta de cruces y de ocupación solo cuenta las pendientes creadas en los últimos 15 minutos, las respuestas muestran el estado EXPIRADA, y pagarla o cancelarla responde 409. Así la reserva vence justo a tiempo sin una tarea programada, y las bases creadas antes de esta regla no necesitaron cambios. Cada reserva pendiente incluye además *pagarAntesDe*, el límite para pagar, que la página le muestra al cliente. Antes de cancelar una reserva confirmada, la página pide confirmación y advierte que el adelanto no se devuelve.

Usuarios y funcionalidades

La solución tiene dos tipos de usuario. El cliente inicia sesión, consulta qué bloques de 30 minutos de un equipo están libres en un día, reserva, paga el adelanto y cancela una reserva. El

administrador inicia sesión y cambia el catálogo, por ejemplo, pone un equipo fuera de servicio o cambia una tarifa. La Tabla 2 relaciona los requisitos funcionales de la guía del curso con el lugar donde la solución los cumple.

Tabla 2

Requisitos funcionales y dónde se cumplen

Código	Requisito	Implementación
RF-01	Resolver el problema definido por el equipo	Reserva en línea de los equipos de Game Spot
RF-02	Proveedor 1 con una capacidad del negocio	proveedor-catalogo: equipos, tarifas y horario
RF-03	Proveedor 2 con otra capacidad	proveedor-reservas: reservas y autenticación
RF-04	Consumidora conectada a los dos proveedores	CatalogoCliente y ReservasCliente con RestClient
RF-05	Al menos dos endpoints por proveedor	8 endpoints en el proveedor 1 y 7 en el proveedor 2
RF-06	GET, POST y PUT, PATCH o DELETE	Se usan los cinco métodos
RF-07	Registrar información desde la interfaz	Reserva guiada y formulario de ingreso
RF-08	Consultar y mostrar datos en la interfaz	Equipos, bloques de disponibilidad y reservas del cliente
RF-09	Actualizar, cancelar o gestionar información	Pagar el adelanto, cancelar, poner un equipo fuera de servicio
RF-10	Integrar los dos proveedores en una operación	Disponibilidad diaria y creación de la reserva
RF-11	Validar datos y mostrar errores	Bean Validation, reglas del negocio y mensajes en la interfaz
RF-12	Probar un proveedor detenido	Respuesta 503 y página que sigue funcionando (Figuras 7 y 8)
RF-13	CRUD sobre recursos del proyecto	CRUD completo de equipos; cancelación lógica de reservas
RF-14	Autenticación JWT y una operación protegida	Ingreso en el proveedor 2; token obligatorio para reservar y cambiar el catálogo
RF-15	Persistencia que sobrevive a un reinicio	Bases H2 en archivo; verificado reiniciando los proveedores

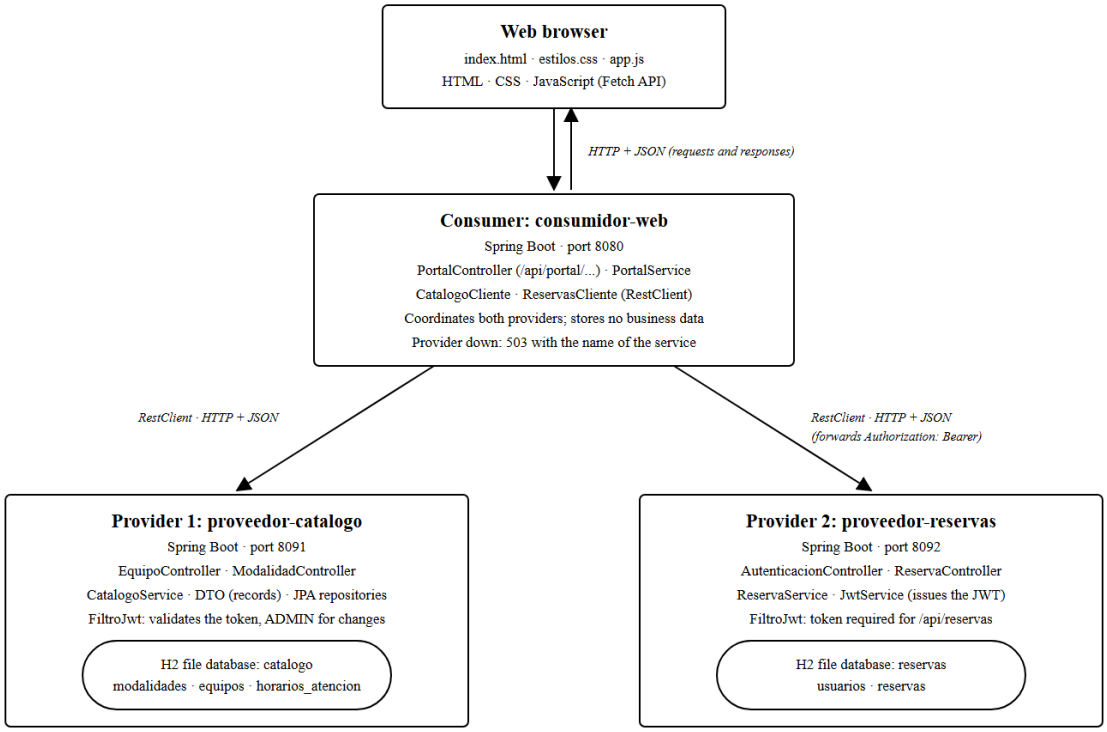
Arquitectura orientada a servicios

La Figura 1 muestra las tres aplicaciones. Cada una es un proyecto Maven aparte, con su propio *pom.xml*, proceso, puerto y configuración. El navegador solo habla con la consumidora, y

la consumidora solo habla con los proveedores a través de sus interfaces HTTP: nunca lee ni escribe en las bases de los proveedores. Las URL de los proveedores son propiedades de la consumidora (*app.proveedores.catalogo.url* y *app.proveedores.reservas.url*) que se pueden cambiar con las variables de entorno CATALOGO_URL y RESERVAS_URL.

Figura 1

Diagrama SOA de Game Spot



Nota. Las flechas indican el sentido de las solicitudes; cada respuesta vuelve por el mismo camino.

Tabla 3

Responsabilidades y datos de cada aplicación

Aplicación	Responsabilidad	Datos propios
proveedor-catalogo (8091)	Lista equipos, tarifas y horario; crea, actualiza y elimina equipos; cambia tarifas. Valida el JWT y exige el rol ADMIN para los cambios.	modalidades, equipos y horarios_atencion en la base H2 catalogo
proveedor-reservas (8092)	Inicia sesión y emite el JWT; crea reservas con las reglas de bloques, precio y cruces; lista reservas y la ocupación de un día; paga y cancela.	usuarios y reservas en la base H2 reservas
consumidor-web (8080)	Sirve la página web; llama a los dos proveedores con RestClient; coordina la disponibilidad y la reserva; traduce las fallas de los proveedores en mensajes claros.	Ninguno; no guarda datos del negocio

Contrato de la API

El contrato de un servicio es lo que promete a sus clientes: la ruta y el método de cada operación, el JSON que recibe y devuelve, si pide token y los códigos de estado que responde. Las Tablas 4 a 6 listan los endpoints. Todos intercambian JSON y todos los errores tienen el mismo cuerpo: la fecha y hora, el código de estado, un mensaje en español y una lista de detalles, por ejemplo uno por cada campo inválido.

Tabla 4

Endpoints del proveedor 1 (catálogo, puerto 8091)

Método	Ruta	Token	Respuestas
GET	/api/equipos	No	200 lista de equipos
GET	/api/equipos/{id}	No	200 equipo; 404 no existe
POST	/api/equipos	ADMIN	201 creado; 400 datos inválidos; 404 modalidad inexistente; 409 nombre repetido
PUT	/api/equipos/{id}	ADMIN	200 actualizado; 400; 404; 409
DELETE	/api/equipos/{id}	ADMIN	204 eliminado; 404
GET	/api/modalidades	No	200 modalidades con su tarifa por hora y por media hora

Método	Ruta	Token	Respuestas
PUT	/api/modalidades/{id}	ADMIN	200 actualizada; 400 tarifas fuera de 1 a 1.000.000; 404
GET	/api/horarios	No	200 horario de los siete días

Nota. Las rutas que piden token responden 401 sin token o con uno inválido, y 403 con un token de cliente.

Tabla 5

Endpoints del proveedor 2 (reservas, puerto 8092)

Método	Ruta	Token	Respuestas
POST	/api/auth/login	No	200 token; 400 datos inválidos; 401 credenciales incorrectas
POST	/api/reservas	Sí	201 reserva pendiente de pago; 400 bloque inválido; 409 bloque ocupado
GET	/api/reservas/mias	Sí	200 reservas del usuario, de la más reciente a la más antigua
GET	/api/reservas/{id}	Sí	200 reserva; 403 reserva de otro usuario; 404
GET	/api/reservas/ocupacion?equipoId=&fecha=	No	200 intervalos ocupados del día, sin datos del cliente
PATCH	/api/reservas/{id}/pago	Sí	200 confirmada; 409 no está pendiente o venció
PATCH	/api/reservas/{id}/cancelacion	Sí	200 cancelada; 403; 409 ya estaba cancelada o venció

Tabla 6

Endpoints de la consumidora (puerto 8080)

Método	Ruta	Proveedores	Respuestas
GET	/api/portal/estado	Ambos	200 disponibilidad de cada proveedor
GET	/api/portal/equipos	Catálogo	200; 503 si el catálogo está caído
GET	/api/portal/horarios	Catálogo	200 horario ordenado por día; 503
GET	/api/portal/equipos/{id}/disponibilidad	Ambos	200 bloques del día con su estado
POST	/api/portal/login	Reservas	200 token; 401
POST	/api/portal/reservas	Ambos	201; 400 fuera del horario; 401; 409; 503
GET	/api/portal/reservas/mias	Reservas	200

Método	Ruta	Proveedores	Respuestas
PATCH	/api/portal/reservas/{id}/pago y /cancelacion	Reservas	200; 409
PATCH	/api/portal/equipos/{id}/estado	Catálogo	200; 401; 403

Ejemplo de intercambio

La consumidora crea una reserva en el proveedor 2 con la siguiente solicitud, después de leer el nombre del equipo y sus dos tarifas en el proveedor 1. La reserva dura 90 minutos, así que su precio es una hora a 5.000 más media hora a 3.000:

```
POST http://localhost:8092/api/reservas
Authorization: Bearer eyJhbGciOiJIUzM4NCJ9...
{ "equipoId": 2, "nombreEquipo": "Xbox Series S #1",
  "tarifaHoraPesos": 5000, "tarifaMediaHoraPesos": 3000,
  "inicio": "2026-11-19T00:00:00Z", "duracionMinutos": 90 }

201 Created
{ "id": 70, "equipoId": 2, "nombreEquipo": "Xbox Series S #1",
  "inicio": "2026-11-19T00:00:00Z", "fin": "2026-11-19T01:30:00Z",
  "precioTotalPesos": 8000, "adelantoPesos": 4000,
  "estado": "PENDIENTE_PAGO", "emailCliente": "laura@gamespot.demo",
  "pagarAntesDe": "2026-11-12T01:19:00Z" }
```

Objetos de transferencia de datos (DTO)

Todos los cuerpos de solicitud y de respuesta son *records* de Java, y los de entrada llevan anotaciones de Bean Validation. Los DTO ocultan las entidades: el cliente nunca ve la estructura de la base, solo los campos del contrato. La Tabla 7 muestra los principales.

Tabla 7

Principales objetos de transferencia de datos

DTO	Aplicación	Campos y tipos	Uso
EquipoRequest	Catálogo	nombre: String (3 a 60); modalidadId: Long; estaDisponible: Boolean	Crear o actualizar un equipo
EquipoResponse	Catálogo	id: Long; nombre: String; modalidadId: Long; modalidad: String; tarifaHoraPesos, tarifaMediaHoraPesos: int; estaDisponible, sePuedeReservar: boolean	Equipo que devuelve la API

DTO	Aplicación	Campos y tipos	Uso
TarifaRequest	Catálogo	tarifaHoraPesos, tarifaMediaHoraPesos: Integer (1 a 1.000.000); estaActiva: Boolean	Cambiar las tarifas de una modalidad
HorarioResponse	Catálogo	diaSemana: short; horaApertura, horaCierre: LocalTime	Horario de un día
LoginRequest	Reservas	email: String (e-mail); password: String	Iniciar sesión
TokenResponse	Reservas	token, tipo, nombre, email, rol: String; expiraEn: Instant	Token emitido
CrearReservaRequest	Reservas	equipoId: Long; nombreEquipo: String; tarifaHoraPesos, tarifaMediaHoraPesos: Integer; inicio: Instant; duracionMinutos: Integer (30 a 240)	Crear una reserva
ReservaResponse	Reservas	id, equipoId: Long; nombreEquipo: String; inicio, fin: Instant; precioTotalPesos, adelantoPesos: int; estado (incluye EXPIRADA); emailCliente: String; pagarAntesDe: Instant	Reserva que devuelve la API
ReservaPortalRequest	Consumidora	equipoId: Long; fecha: LocalDate; hora: LocalTime; duracionMinutos: Integer	Formulario de reserva de la página
DisponibilidadResponse	Consumidora	equipo; fecha: LocalDate; atiende: boolean; bloques: lista de {inicio, fin, estaLibre, motivo}	Bloques de disponibilidad
ApiError	Todas	timestamp: Instant; status: int; mensaje: String; detalles: List<String>	Toda respuesta de error

Diseño de las aplicaciones

Capas Controller, Service y Repository

Las tres aplicaciones siguen la misma estructura por capas. Los controladores, como *EquipoController*, *ReservaController* y *PortalController*, asocian rutas y métodos HTTP, validan la entrada con *@Valid* y delegan en un servicio; no contienen reglas del negocio. Los servicios, como *CatalogoService*, *ReservaService* y *PortalService*, tienen las reglas y se ejecutan dentro de transacciones. En los proveedores, los servicios leen y escriben datos solo a través de repositorios de Spring Data JPA, que cumplen el papel del patrón DAO: *EquipoRepository*, por ejemplo, declara *existsByNombreIgnoreCase*, y *ReservaRepository* declara la consulta JPQL *buscarActivasQueSeCruzan*, que encuentra las reservas activas de un equipo que se cruzan con

un intervalo (*r.inicio < :hasta and r.fin > :desde*): las confirmadas y las pendientes que siguen dentro de sus 15 minutos para pagar. Un manejador global de excepciones en cada aplicación convierte las excepciones en respuestas *ApiError* con el código correcto.

RestClient en la consumidora

La consumidora define un RestClient por proveedor en *ClientesConfig*, con la URL base del proveedor, un tiempo máximo de conexión de 2 segundos y uno de lectura de 5 segundos. *CatalogoCliente* y *ReservasCliente* envuelven cada llamada: arman la solicitud, reenvían el encabezado *Authorization* del navegador cuando la operación está protegida y convierten la respuesta JSON en un DTO. Una ayuda compartida, *LlamadaProveedor*, traduce las fallas: un error de conexión o un tiempo agotado se vuelve una respuesta 503 que nombra el servicio caído, y una respuesta de error de un proveedor llega al navegador con el mismo código y el mismo mensaje.

Operaciones integradas

La consumidora coordina los dos proveedores en dos operaciones. Para mostrar la disponibilidad de un equipo en un día, lee el equipo y el horario en el proveedor 1 y los intervalos ocupados en el proveedor 2, y marca cada bloque de 30 minutos como libre, ocupado, pasado o fuera de servicio. Para crear una reserva sigue estos pasos: (a) el navegador envía el equipo, la fecha, la hora de inicio y la duración a POST /api/portal/reservas con el token; (b) la consumidora lee el equipo en el proveedor 1 y la rechaza con 409 si está fuera de servicio; (c) lee el horario en el proveedor 1 y rechaza la reserva con 400 si no cabe entre las 3:00 p. m. y las 9:00 p. m.; (d) envía al proveedor 2 el nombre del equipo, sus dos tarifas vigentes y la hora de inicio convertida desde la hora de Bogotá, con el mismo token; (e) el proveedor 2 valida el bloque,

revisa los cruces, calcula el precio y el adelanto del 50 % y guarda la reserva; y (f) la consumidora devuelve la reserva al navegador.

Interfaz web

La consumidora sirve *index.html*, *estilos.css* y *app.js* como recursos estáticos. El script usa la API Fetch para cada solicitud a */api/portal*, guarda el token en el almacenamiento de sesión del navegador después de iniciar sesión y lo agrega a las solicitudes protegidas. La página muestra el horario, los equipos con sus dos tarifas (Figuras 2 y 3), una reserva guiada en tres pasos —consola, día y hora, y confirmación— (Figuras 4 y 5), las reservas del cliente con botones para pagar el adelanto y cancelar (Figura 6) y una tabla de administración para el administrador. También incluye contenido del local que no viene de los proveedores: el catálogo de juegos de cada consola (*datos/juegos.json*), videos del local, una versión de práctica del «Reto 10 segundos» de los fines de semana y los torneos ya realizados. Los mensajes de error del backend se muestran tal como llegan, en español, y una barra de estado indica si cada proveedor está disponible.

Modelo de datos y persistencia

DDL

Cada proveedor define sus tablas en *schema.sql*, que Spring Boot ejecuta al arrancar. Las sentencias usan *create table if not exists*, así que una base existente conserva sus datos. El proveedor 1 tiene *modalidades* (llave de identidad, nombre único, tarifa por hora y tarifa por media hora entre 1 y 1.000.000, y un indicador de activa), *equipos* (llave de identidad, nombre único, llave foránea *modalidad_id* hacia *modalidades* y un indicador de disponibilidad) y *horarios_atencion* (día de la semana del 1 al 7 como llave, y horas de apertura y cierre con una restricción de que el cierre sea posterior). El proveedor 2 tiene *usuarios* (llave de identidad,

correo único, hash BCrypt de la contraseña y un rol limitado a CLIENTE o ADMIN) y *reservas* (llave de identidad, llave foránea *usuario_id*, id y nombre del equipo, inicio, fin, precio, adelanto, estado limitado a PENDIENTE_PAGO, CONFIRMADA o CANCELADA (EXPIRADA se calcula, no se guarda), fechas de creación y cancelación, y restricciones de que el fin sea posterior al inicio y el adelanto no supere el precio). El id del equipo en *reservas* no tiene llave foránea porque los equipos viven en otra aplicación con su propia base.

DML y CRUD

Cada proveedor carga sus datos iniciales con *data.sql*. El proveedor 1 inserta los cinco equipos reales, las tres modalidades con sus tarifas por hora y por media hora y el horario de los siete días (3:00 p. m. a 9:00 p. m.) con *merge into*, y el proveedor 2 inserta tres usuarios de prueba. Cada inserción se ejecuta solo si la fila no existe, así que reiniciar la aplicación no duplica datos ni deshace cambios hechos por la API; una corrección de una sola vez actualiza las bases creadas antes de que existiera la tarifa de media hora. La Tabla 8 muestra cómo cada operación CRUD de la API corresponde a SQL.

Tabla 8

Correspondencia entre CRUD, HTTP y SQL

CRUD	**Endpoint HTTP**	**SQL que ejecuta JPA**
Crear	POST /api/equipos; POST /api/reservas	INSERT INTO equipos ...; INSERT INTO reservas ...
Consultar	GET /api/equipos; GET /api/reservas/mias	SELECT ... FROM equipos JOIN modalidades ...; SELECT ... FROM reservas WHERE usuario ...
Actualizar	PUT /api/equipos/{id}; PUT /api/modalidades/{id}; PATCH /api/reservas/{id}/pago	UPDATE equipos SET ...; UPDATE modalidades SET ...; UPDATE reservas SET estado = ...
Eliminar	DELETE /api/equipos/{id}	DELETE FROM equipos WHERE id = ?
Eliminación lógica	PATCH /api/reservas/{id}/cancelacion	UPDATE reservas SET estado = ’CANCELADA’, cancelada_en = ...

Nota. Las reservas se cancelan en lugar de borrarse porque el local necesita conservar el historial de lo reservado y lo pagado, incluidos los adelantos que no se devuelven; una reserva cancelada libera su bloque.

ORM

Los proveedores usan JPA con Hibernate para relacionar entidades y tablas. La entidad *Equipo*, por ejemplo, lleva *@Entity* y *@Table(name = "equipos")*; su campo *id* usa *@GeneratedValue(strategy = IDENTITY)*; y su campo *modalidad* usa *@ManyToOne* con *@JoinColumn(name = "modalidad_id")*, así que al cargar un equipo también se cargan su modalidad y sus tarifas. Las enumeraciones como *EstadoReserva* se guardan como texto con *@Enumerated(EnumType.STRING)*. Hibernate trabaja con *ddl-auto: validate*: no crea tablas, solo comprueba al arrancar que las entidades coincidan con las tablas que crea *schema.sql*, que es la única fuente de verdad del esquema.

Mecanismo de persistencia

Cada proveedor usa una base H2 guardada en un archivo (*./data/catalogo* y *./data/reservas*, relativos al directorio de trabajo de la aplicación), como lo permite la guía del curso, así que los datos sobreviven a un reinicio y no hace falta un servidor de base de datos. El equipo lo verificó creando y pagando una reserva, deteniendo los dos proveedores, arrancándolos de nuevo y encontrando la reserva todavía confirmada. Las pruebas automatizadas usan bases H2 en memoria creadas con los mismos scripts. La versión monolítica del proyecto usa PostgreSQL en Supabase; la versión orientada a servicios no la usa.

Autenticación y autorización con JWT

El proveedor 2 autentica a los usuarios. POST /api/auth/login compara la contraseña con su hash BCrypt y devuelve un JSON Web Token (Jones et al., 2015) firmado con HMAC-SHA.

El token lleva el correo como sujeto, el rol y el nombre como *claims* y un vencimiento de dos horas. El navegador guarda el token y lo envía como *Authorization: Bearer* en cada solicitud protegida; la consumidora lo reenvía sin cambios al proveedor dueño de la operación.

Los dos proveedores validan los tokens con un filtro de servlet y el mismo secreto, que se lee de la variable de entorno JWT_SECRETO y nunca se escribe en el repositorio. En el proveedor 2, toda ruta bajo */api/reservas* pide un token válido, excepto la consulta pública de ocupación, y el servicio además comprueba que un cliente solo consulte, pague o cancele sus propias reservas (403 si no). En el proveedor 1 las consultas son públicas, y crear, actualizar o eliminar pide un token con rol ADMIN. Una solicitud sin token o con uno inválido o vencido recibe 401, y un token válido sin el rol necesario recibe 403. La consumidora no valida tokens por su cuenta; confía en los proveedores, que son los dueños de los recursos protegidos.

Plan de pruebas y resultados

El plan de pruebas tiene tres niveles. Primero, cada proyecto tiene pruebas de integración automatizadas que cubren cada criterio de aceptación y se ejecutan con *./mvnw test*. Segundo, una colección de Postman ejercita las aplicaciones reales por HTTP. Tercero, una prueba manual de punta a punta usa la página web, incluso con un proveedor detenido. Cada caso automatizado tiene un código (CP-36 para el catálogo, CP-37 para las reservas y CP-38 para la consumidora) que también aparece en el nombre de la prueba.

Pruebas automatizadas

Las pruebas de los proveedores levantan la aplicación completa con *@SpringBootTest* y una base H2 en memoria, llaman a los endpoints con MockMvc y tokens reales, y deshacen cada prueba con *@Transactional*. Las pruebas de la consumidora reemplazan a los proveedores por dobles de Mockito para comprobar la lógica de coordinación, y otra clase usa un servidor HTTP

simulado y un puerto cerrado para comprobar las llamadas de RestClient, el reenvío del token, la traducción de los errores y el comportamiento con un proveedor detenido. Las Tablas 9 y 10 las resumen. El 11 de noviembre de 2026 las 29 pruebas pasaron sin fallos.

Tabla 9

Pruebas automatizadas por proyecto

Proyecto	Clases de prueba	Pruebas	Resultado
proveedor-catalogo	CatalogoTests (CP-36a a CP-36h)	8	0 fallos
proveedor-reservas	ReservasTests (CP-37a a CP-37k)	11	0 fallos
consumidor-web	PortalTests y ClientesHttpTests (CP-38a a CP-38j)	10	0 fallos
Total		29	0 fallos

Tabla 10

Casos de prueba automatizados

Caso	Qué comprueba
CP-36a	Lista los cinco equipos reales con su modalidad, sus dos tarifas y su estado
CP-36b	Consulta un equipo por id y responde 404 si no existe
CP-36c	CRUD completo de equipos con token de administrador
CP-36d	Valida los datos (400) y no permite nombres repetidos (409)
CP-36e	Sin token o con token inválido responde 401; un cliente recibe 403
CP-36f	Cambia las tarifas de una modalidad y los equipos las reflejan
CP-36g	Entrega el horario de los siete días, todos de 3:00 a 9:00 p. m.
CP-36h	data.sql corrige una base antigua una sola vez y luego respeta los cambios hechos por la API
CP-37a	El ingreso devuelve un JWT con el rol; credenciales incorrectas dan 401
CP-37b	Crea la reserva en PENDIENTE_PAGO con el precio y un adelanto del 50 %
CP-37c	Bloques desalineados, duraciones inválidas o en el pasado dan 400
CP-37d	Un bloque que se cruza con otra reserva activa del mismo equipo da 409
CP-37e	Cada cliente ve solo sus reservas; la ocupación es pública y sin datos del cliente
CP-37f	Pagar confirma la reserva; pagarla otra vez da 409
CP-37g	Cancelar es lógico y libera el bloque; otro cliente recibe 403
CP-37h	Las operaciones protegidas sin token o con token inválido dan 401

Caso	Qué comprueba
CP-37i	Precio: cada hora a la tarifa por hora y la media hora que sobra a su propia tarifa
CP-37j	Una reserva sin pagar vence a los 15 minutos: se lista como EXPIRADA, libera el bloque y pagarla o cancelarla da 409
CP-37k	Dentro de los 15 minutos la reserva sigue pendiente, informa su límite de pago y bloquea el turno; pagada ya no vence
CP-38a	Lista los equipos del proveedor de catálogo
CP-38b	La disponibilidad cruza el horario del catálogo con la ocupación de reservas
CP-38c	Reservar toma el nombre y las dos tarifas del catálogo, crea la reserva en el proveedor 2 y devuelve el límite de pago
CP-38d	Fuera del horario 400, equipo fuera de servicio 409, datos inválidos 400, sin sesión 401
CP-38e	Con el catálogo detenido, la consumidora responde 503 y sigue atendiendo lo demás
CP-38f	La página web se sirve desde la consumidora
CP-38g	RestClient pide GET /api/equipos al catálogo y lee el JSON
CP-38h	El token se reenvía al proveedor 2 y su 409 llega con su mensaje
CP-38i	Con un proveedor detenido (puerto cerrado), la llamada se vuelve un 503
CP-38j	El horario sale ordenado por día y da 503 con el catálogo detenido

Postman

La colección *soa/postman/GameSpot-SOA.postman_collection.json* tiene 40 solicitudes en cinco carpetas: autenticación, proveedor 1, proveedor 2, consumidora y proveedor detenido. Guarda los tokens y los identificadores creados en variables de la colección y calcula la fecha de mañana en hora de Bogotá antes de las reservas, así que se puede ejecutar varias veces: borra el equipo que crea, cancela sus reservas y vuelve a habilitar el equipo que pone fuera de servicio. Cada solicitud tiene una prueba que revisa su código de estado, y algunas también revisan valores, como el adelanto de 2.500 pesos de una reserva de una hora y el límite de pago de una reserva nueva. Las cuatro primeras carpetas se ejecutan con las tres aplicaciones encendidas, y la quinta después de detener el proveedor 1.

Prueba de punta a punta

El 11 de noviembre de 2026 el equipo arrancó las tres aplicaciones y usó la página web como cliente. La Figura 2 muestra la portada con el horario y las cifras leídas de los proveedores, y la Figura 3 el cliente con sesión iniciada y los equipos leídos del proveedor 1 con sus dos tarifas. En la reserva guiada, el cliente eligió la Xbox Series S #1, el día siguiente y 90 minutos; la Figura 4 muestra los bloques libres de 3:00 p. m. a 8:30 p. m. y el precio de 8.000 pesos con un adelanto de 4.000. La Figura 5 muestra la reserva creada con la operación integrada y su plazo de 15 minutos para pagar. La Figura 6 la muestra confirmada después de pagar el adelanto, junto a una segunda reserva que sigue pendiente y muestra la hora límite para pagarla; al cancelar la confirmada, la página pide confirmación y advierte que el adelanto de 4.000 pesos no se devuelve.

Figura 2

Portada con el horario y las cifras del local

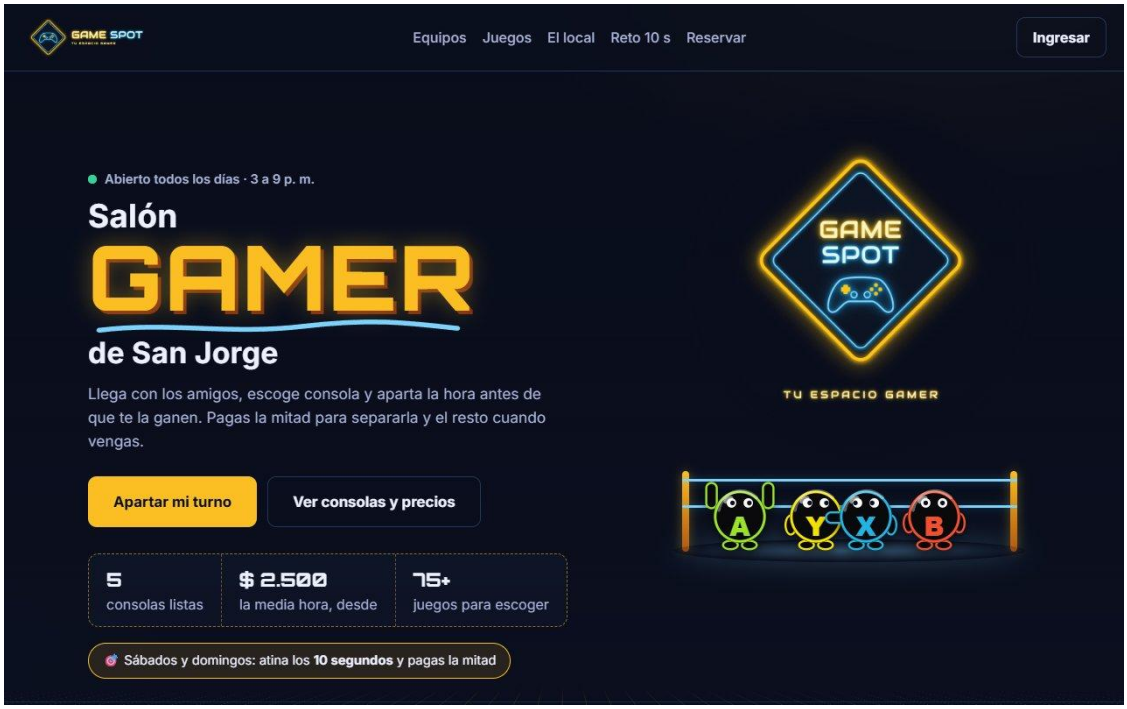


Figura 3

Cliente con sesión iniciada y equipos del proveedor 1



Figura 4

Disponibilidad de un día: horario (proveedor 1) y ocupación (proveedor 2)

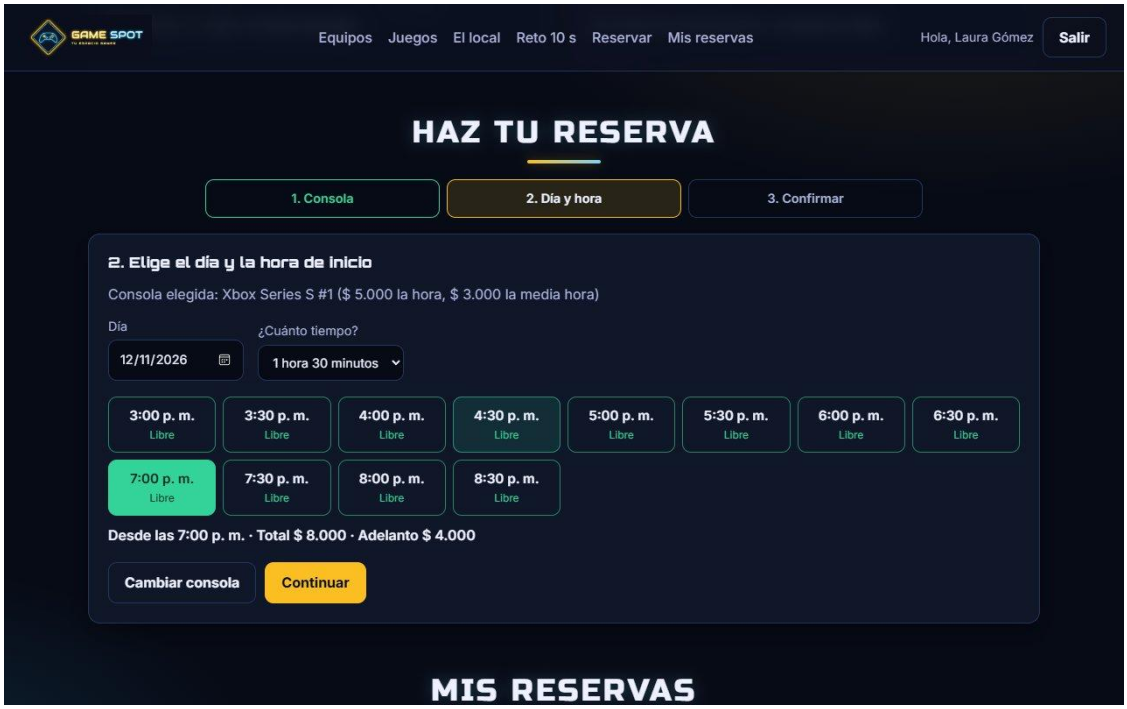


Figura 5

Reserva creada con los dos proveedores

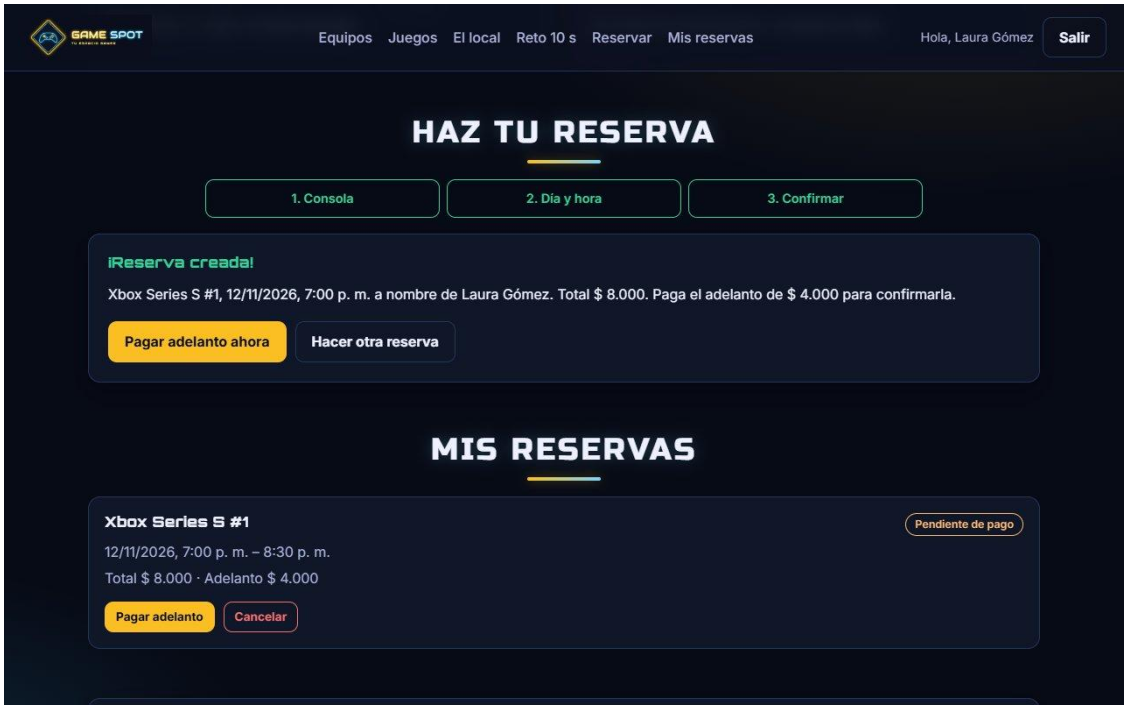
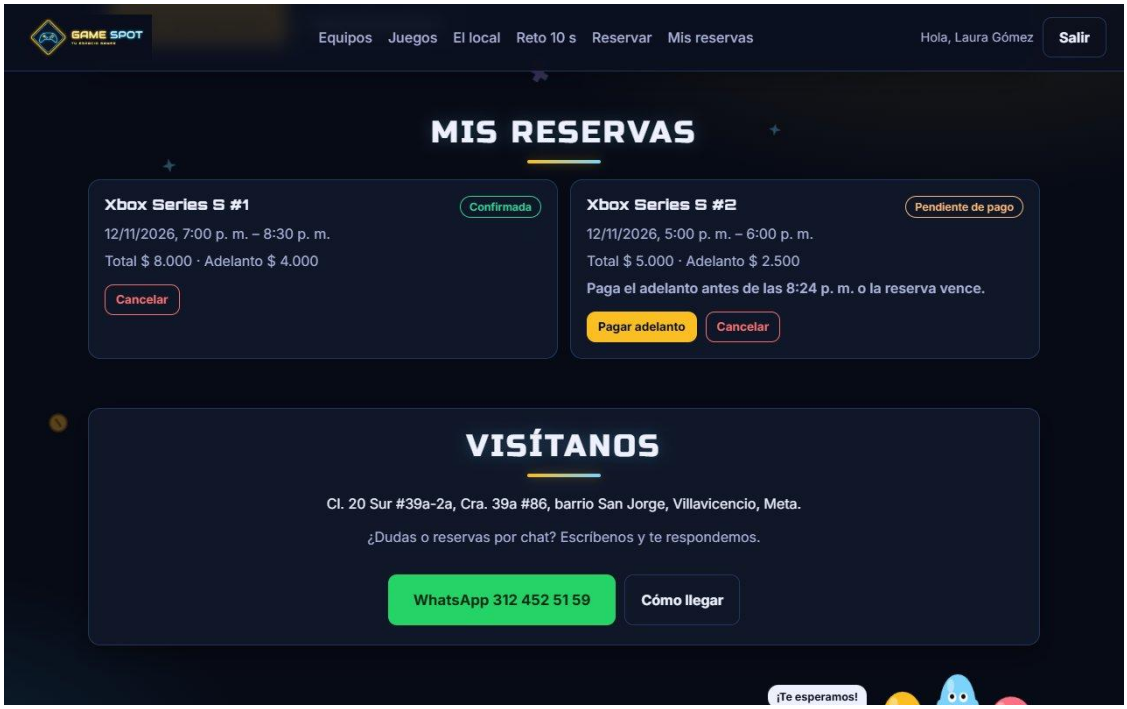


Figura 6

Reservas del cliente: una confirmada y otra pendiente con su límite de pago



Proveedor detenido

Con las tres aplicaciones en marcha, el equipo detuvo el proveedor 1. Como muestran las Figuras 7 y 8, la página siguió funcionando: la sección de equipos muestra el mensaje que devolvió la consumidora, que respondió a GET /api/portal/equipos con el código 503 y el texto de que el servicio de catálogo no está disponible, en lugar de un error sin manejar; y las reservas del cliente, que vienen del proveedor 2, se siguen mostrando y se pueden cancelar.

Figura 7

Sección de equipos con el proveedor de catálogo detenido

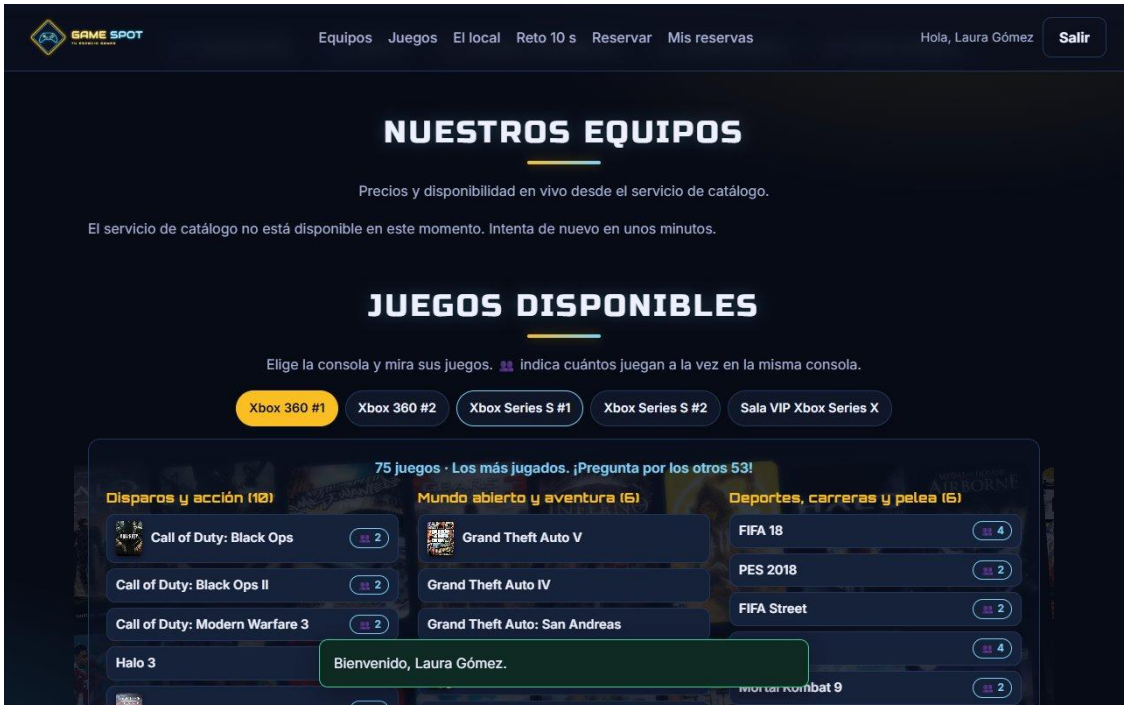
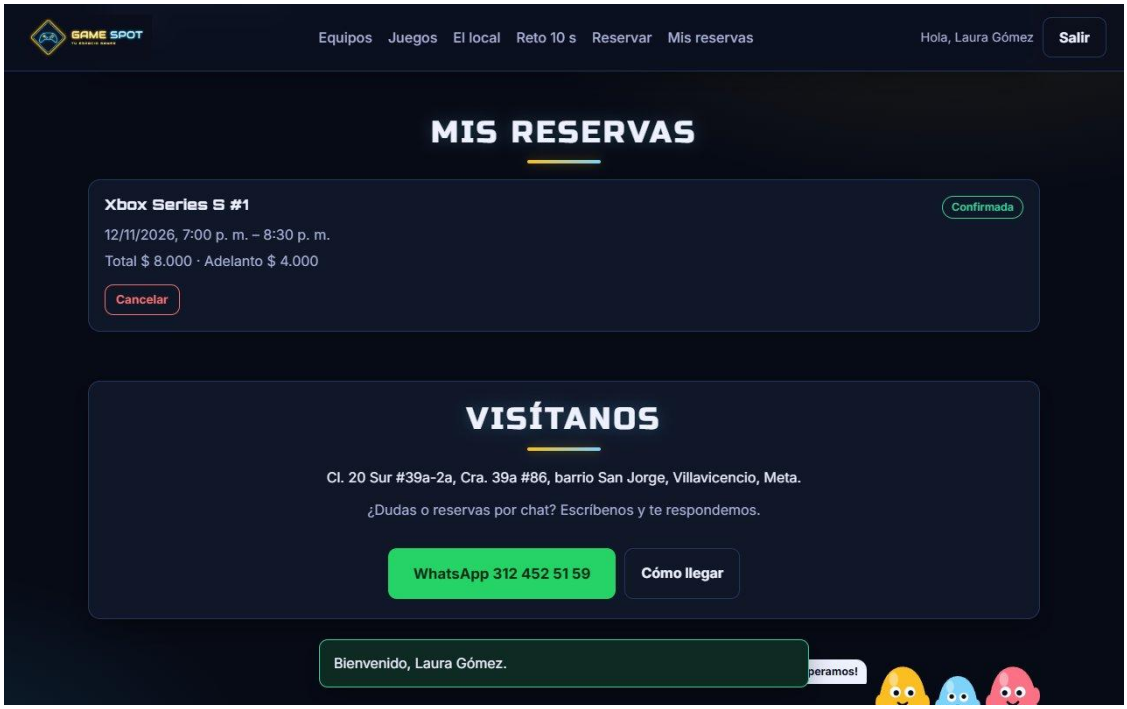


Figura 8

Reservas del cliente disponibles con el catálogo detenido



Guía de ejecución

Las aplicaciones necesitan JDK 21. No hacen falta Docker ni una base de datos externa. Antes de arrancar los proveedores, defina la variable de entorno JWT_SECRETO con el mismo valor de al menos 32 caracteres en los dos, por ejemplo en las configuraciones de ejecución de IntelliJ IDEA. Luego arranque, en este orden, *soa/proveedor-catalogo*, *soa/proveedor-reservas* y *soa/consumidor-web* con *./mvnw spring-boot:run* (o *.\mvnw.cmd* en PowerShell) y abra http://localhost:8080. Los archivos H2 se crean en el directorio de trabajo de cada aplicación, así que no conviene cambiarlo entre ejecuciones. La Tabla 11 resume la configuración de cada aplicación.

Tabla 11

Configuración de cada aplicación

Aplicación	Puerto	Ajustes principales
proveedor-catalogo	8091	JWT_SECRETO; archivo H2 en ./data/catalogo
proveedor-reservas	8092	JWT_SECRETO; archivo H2 en ./data/reservas; tokens válidos por 2 horas
consumidor-web	8080	CATALOGO_URL (por defecto http://localhost:8091); RESERVAS_URL (por defecto http://localhost:8092); tiempos máximos de 2 y 5 segundos

Nota. Cuentas de prueba: admin@gamespot.demo con contraseña AdminDemo123, y laura@gamespot.demo o andres@gamespot.demo con contraseña ClienteDemo123. Son datos de demostración, no credenciales reales.

Conclusiones, dificultades y aprendizajes

La solución cumple la arquitectura mínima del curso: dos proveedores con capacidades distintas, una consumidora que los integra con RestClient, una interfaz web con Fetch, persistencia con DDL, DML, repositorios y ORM, y JWT protegiendo las operaciones que modifican datos. La reserva integrada muestra el principal beneficio del diseño: la consumidora combina el catálogo y las reservas sin que ningún proveedor conozca al otro, y una falla en un proveedor solo afecta a las funciones que dependen de él.

La principal dificultad fue que el equipo partió de un sistema monolítico con una sola base de datos que hacía cumplir las reglas de reserva con restricciones y disparadores. Dividirlo obligó a decidir qué aplicación es dueña de cada dato y a llevar algunas reglas a los servicios, como la revisión de cruces, que el monolito hacía cumplir con una restricción de exclusión de PostgreSQL que H2 no ofrece. Una segunda dificultad fue la seguridad entre aplicaciones: los dos proveedores debían validar los mismos tokens sin llamarse entre sí, lo que se resolvió con un secreto compartido que llega por una variable de entorno. Una tercera fue hacer visibles las fallas

para el usuario, lo que exigió tiempos máximos en RestClient y un cuerpo de error uniforme en las tres aplicaciones. Por último, la revisión del avance mostró que algunas reglas del negocio seguían abiertas; el equipo fijó un único horario, implementó el adelanto sin devolución y el vencimiento de 15 minutos, y dejó el tiempo adicional para después del flujo principal.

El equipo aprendió que el contrato de un servicio debe definirse antes de construir la consumidora, que las reglas del negocio deben cerrarse con el cliente antes de programarlas, que una consumidora debe tratar cada llamada remota como algo que puede fallar, y que copiar los datos que un servicio necesita, como el nombre y las tarifas del equipo en una reserva, mantiene a los servicios independientes a costa de algo de duplicación.

Referencias

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, Universidad de California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

H2 Database Engine. (s. f.). *H2 database engine*. Recuperado el 10 de noviembre de 2026 de https://www.h2database.com/html/main.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Postman. (s. f.). *Postman learning center*. Recuperado el 10 de noviembre de 2026 de https://learning.postman.com/docs/

Spring. (s. f.). *REST clients*. Spring Framework reference documentation. Recuperado el 10 de noviembre de 2026 de https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

Apéndice

Declaración sobre el uso de inteligencia artificial

El equipo usó un asistente de programación con inteligencia artificial (Claude, de Anthropic) como apoyo para escribir código, pruebas, la colección de Postman y este documento. El equipo definió la idea del negocio, la división en servicios, las reglas del negocio y las decisiones de diseño, revisó cada pull request y verificó los resultados aquí reportados ejecutando las pruebas, la colección de Postman y las aplicaciones.